



Security Review Report for LEDiG Technologies

September 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
 - Security Review Lead
 - Scope
 - Changelog
4. Severity Structure
 - Severity characteristics
 - Issue symbolic codes
5. Findings Summary
6. Weaknesses
 - Strike Blocklist Writer Can Prevent Option Exercise
 - Partial exercise() unfairly assigns the cheapest writer first
 - totalSoldBase redundantly stores derivable state
 - Limited strike-price range across token decimals
 - Redundant AccessControl in FeeRouter

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This audit covers Ledig Protocol, an on-chain, fully collateralised stablecoin call-options protocol for emerging-market foreign-exchange use cases. It is designed for institutions that want to lock access to a USD stablecoin against another stablecoin at a fixed rate before a stated expiry.

The review was conducted over a week and included a comprehensive analysis of the relevant smart contracts.

During the assessment, we did not identify any major issues. We identified one low-severity finding and four informational issues.

All identified issues were either remediated or acknowledged by the development team and subsequently verified by our auditors.

Following the remediation phase, we conclude that the protocol's overall security posture and code quality have improved as a result of this audit.

3. Security Review Details

- Review Led by

Trung Dinh, Lead Security Researcher

- Scope

The analyzed resources are located on:

<https://github.com/Ledig-Tech/ledig-protocol/tree/84551a1e1aa47f95b6582624fee0d8058260a046>

The issues described in this report were fixed in the following commit:

[2306917b8c33e6ae312d4df63518e3043c0e9232](#)

- Changelog

■ 7 September 2026	Audit start
■ 14 September 2026	Initial report
■ 22 September 2026	Revision received
■ 23 September 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

- 1. Impact of the vulnerability
- 2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical	Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.
High	Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

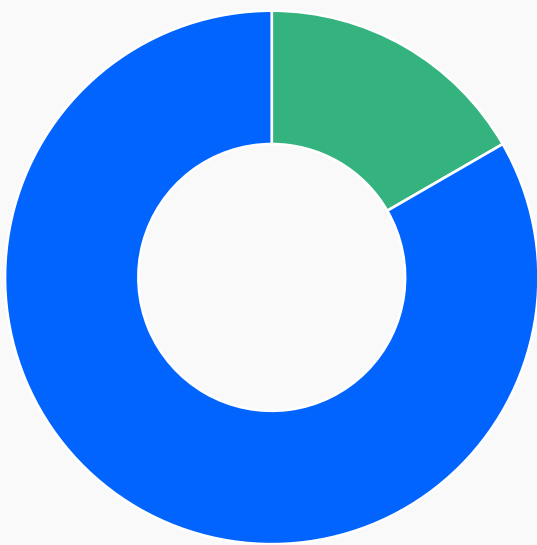
Severity

Number of findings

Critical	0
High	0
Medium	0
Low	1
Informational	4

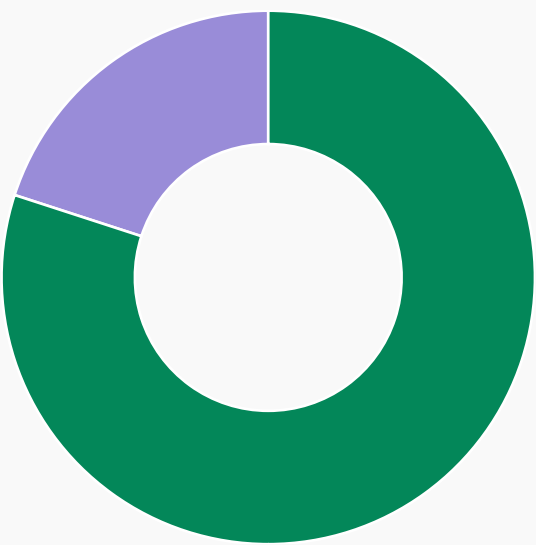
Total:

5



Low

Informational



Fixed

Acknowledged

6. Weaknesses

This section contains the list of discovered weaknesses.

LEDIG1-1 | Strike Blocklist Writer Can Prevent Option Exercise

Fixed 

Severity:

Low

Probability:

Rare

Impact:

Medium

Path:

src/OptionSeriesV2.sol#L805

Description:

The **OptionSeriesV2.exercise()** function allows the buyer to exercise their option. It iterates through all reserved writers in the order of the **reservedWritersByBuyer** array (first-fill, first-served) and settles each writer atomically. During settlement, the buyer transfers the Strike asset to the writer, while the Base asset is transferred from the vault to the writer.

```
function exercise(
    uint256 amountBase
) external nonReentrant returns (uint256 payout) {
    ...
    address[] storage ws = reservedWritersByBuyer[msg.sender];
    ...

    for (uint256 i = 0; i < ws.length && remaining > 0; i++) {
        ...
    }

    ...

    for (uint256 i = 0; i < settlementCount; i++) {
        ...
        _transferFromExact(strike, msg.sender, w, strikeSlice);
        ...
    }
}
```

The issue arises when one of the reserved writers is blocklisted by the Strike token. For example, if USDC is used as the Strike asset and a reserved writer is blacklisted by USDC, the Strike transfer from the buyer to that writer will revert.

Since the entire exercise process is atomic, this revert causes the whole **exercise()** transaction to fail. Moreover, the buyer cannot choose which writer to exercise against, as the function always processes writers according to the **reservedWritersByBuyer** array.

As a result, a single blocklisted writer can prevent the buyer from exercising their option altogether. The buyer may therefore be unable to receive the expected payout despite having purchased the option and paid the premium.

Note that the issue is broader when the Strike token is an ERC777. An attacker can leverage the ERC777 callback mechanism to block the exercise process in a similar manner to the blocklist issue described above.

Furthermore, if the Base token is an ERC777, a malicious writer can also DoS the option purchase process. By placing a position at the smallest price lot, the writer can use the ERC777 callback to revert the token transfer when a buyer attempts to purchase the option, causing the entire purchase transaction to fail.

Remediation:

Instead of transferring the Strike asset directly from the buyer to the writer during exercise, the protocol could first transfer the Strike tokens from the buyer to the **OptionSeries** contract and account for each writer's entitlement in storage. Writers would then claim their accumulated Strike tokens through a separate function.

LEDIG1-2 | Partial exercise() unfairly assigns the cheapest writer first

Acknowledged

Severity:

Informational

Probability:

Unlikely

Impact:

Informational

Path:

./src/OptionSeriesV2.sol

Description:

When a buyer partially exercises, **exercise()** assigns writers sequentially:

```
function exercise(  
    uint256 amountBase  
) external nonReentrant returns (uint256 payout) {  
    --snip--  
    for (uint256 i = 0; i < ws.length && remaining > 0; i++) {  
        -- snip--  
        uint256 take = r;  
        if (take > remaining) take = remaining;
```

The writer list is populated in fill order:

```
reservedWritersByBuyer[buyer].push(writer);
```

Since **buy()** fills the cheapest quotes first, the cheapest writer is normally first in the list.

Example:

- Writer A quotes 1% and provides 100 BASE.
- 10 days later Writer B quotes 5% and provides 100 BASE.
- The buyer buys all 200 BASE, filling both writers.
- The buyer later needs only 100 BASE, so they exercise half.

The result is:

```
Writer A (1%): assigned 100 BASE  
Writer B (5%): assigned 0 BASE
```

So the writer offering the best price earns less premium but takes 100% of the exercise, while the writer charging 5% earns more premium and takes no assignment.

This could happen when a buyer may buy an option for 1,000 BASE but only need 300 BASE today, exercising 300 and leaving the remaining 700 BASE open for later.

Remediation:

Consider distributing partial exercises pro-rata across all backing writers instead of draining writers sequentially.

Commentary from the client:

"We agree that partial exercises should be shared proportionally between backing writers. However, this would require significant changes to the allocation system and accounting. Since this is an informational finding about fairness between writers, we will postpone the change. We need more time to properly assess and test it before changing the existing behaviour."

LEDIG1-3 | totalSoldBase redundantly stores derivable state

Fixed 

Severity:

Informational

Probability:

Rare

Impact:

Informational

Path:

./src/OptionSeriesv2.sol

Description:

`_totalSoldBase` duplicates information already derivable from `params.capBase - _totalRemainingBase`. Since `buy()` already updates `_totalRemainingBase`, maintaining a second counter causes an unnecessary storage write on every purchase:

```
function buy(..) external nonReentrant whenNotPaused returns (...) {  
  --snip-  
  _totalRemainingBase -= amountBase;  
  _totalSoldBase += amountBase;  
}
```

This unnecessary storage write costs ~22,100 gas on the first buy and ~5,000 gas on subsequent buys.

Remediation:

Remove `_totalSoldBase` and derive the value when needed:

```
function totalSoldBase() external view returns (uint256) {  
  return params.capBase - _totalRemainingBase;  
}
```

Likewise, `getSeriesDetails()` can derive `totalSoldBase` from the same values.

LEDIG1-4 | Limited strike-price range across token decimals

Fixed 

Severity:

Informational

Probability:

Rare

Impact:

Informational

Path:

src/OptionSeriesV2.sol

Description:

strikeRate is a uint32 raw-unit multiplier. **STRIKE** quotes are calculated without decimal normalization, so the rate must incorporate the decimal difference between the two tokens.

```
function _strikeAmount(  
    uint256 amountBase,  
    uint32 strikeRate  
) private pure returns (uint256) {  
    return Math.mulDiv(amountBase, uint256(strikeRate), 10_000);  
}
```

For example, with 6-decimal BASE and 18-decimal STRIKE, a price of 1.05 STRIKE per BASE requires **strikeRate = 10,500,000,000,000,000**. This exceeds the **uint32** maximum of **4,294,967,295**.

The bounded rate means some token-pair and strike-price combinations cannot be represented even when decimal scaling is calculated correctly.

Remediation:

If the additional price range is required, widen the rate representation consistently across contracts, interfaces, and deployment tooling for a new deployment generation.

LEDIG1-6 | Redundant AccessControl in FeeRouter

Fixed ✓

Severity:

Informational

Probability:

Rare

Impact:

Informational

Path:

src/FeeRouter.sol#L8

src/FeeRouter.sol#L18

Description:

The **FeeRouter** contract inherits from **AccessControl** and grants the **GOVERNOR_ROLE** to the admin address in its constructor:

```
contract FeeRouter is IFeeRouter, AccessControl {  
    ...  
    constructor(address admin, address recipient, uint16 bps) {  
        ...  
        _grantRole(GOVERNOR_ROLE, admin);  
        ...  
    }  
}
```

However, **FeeRouter** only contains the view function **routePremium()**. Neither **AccessControl** nor **GOVERNOR_ROLE** is used anywhere in the contract.

As a consequence, inheriting **AccessControl** and granting **GOVERNOR_ROLE** add unnecessary code and storage overhead.

Remediation:

Consider removing the **AccessControl** inheritance and the unused **GOVERNOR_ROLE** logic from **FeeRouter**.

hexens x  LEDiG

